Laborversuch 1: Shellskripte (Vorbereitungsteil)

Lernziele:

In diesem Laborversuch lernen Sie, Routineaufgaben auf einem Unix System (Linux, MacOS) mit einem Bash-Skript zu automatisieren. Sie **müssen** sich hierzu vorbereiten indem Sie

- die Bash Cheat Sheets durcharbeiten
- das Skript ***Skript Linux-Crashkurs***, lesen.
- vorhandene Bash-Skripte untersuchen (z.B.: unter `/usr/bin`, mit einem Editor wie VS Code, `less` oder mit `file /usr/bin/*`).

Sie bearbeiten diesen **Vorbereitungsteil eigenständig in der Woche** vor dem Labortermin. Im Labor selbst lösen und präsentieren Sie ohne weitere Hilfe eine kleine Teilaufgabe!

Erwartete Lernziele:

- Sie verstehen den Zusammenhang zwischen der Bash, Bash-Skripten und deren Interaktion mit anderen systemnahen Programmen.
- Sie können selbstständig und ohne Hilfe mit der Bash interagieren und eigene Bash-Skripte entwickeln.
- Sie können Metadaten aus Dateien (hier: ID3-Tags) mit einem externen Kommandozeilenwerkzeug auslesen und in einem Skript weiterverarbeiten.

Erfolgskontrolle:

Um den Versuch testiert zu bekommen, müssen Sie **ein im Labor selbst geschriebenes Shell-Skript präsentieren (Vorbereitungsteil + Laborteil)**. Den Laborteil erhalten Sie am Tag des Labors. Am Ende dieses Labors müssen Sie jeder ihr eigenes Skript in Moodle hochladen und erfolgreich schlüssig erklären können.

Einführung:

MP3 Dateien können sogenannte ID3 Tags[1] enthalten, um bspw. den Titel oder den Interpreten zu kodieren. Diese Tags werden dann vom ihrem Audioplayer interpretiert und ausgegeben.

Mit dem Unix Programm `id3v2` können Sie mit `--list` alle Tags einer Datei ausgeben – id3 Tags der Version 1 und der Version 2. Der Parameter `--help` liefert alle weiteren notwendigen Optionen. Das folgende Bild zeigt beispielhaft die Ausgabe von `id3v2` für die MP3-Datei „42.0.mp3“, einem Projekt von Thomas Wenzlaff, bei dem alle Zahlen von 0 bis 100 mit einer Nachkommastelle ausgesprochen werden und als einzelne MP3-Datei mit entsprechenden ID3-Tags abgelegt wurden:

[1] Siehe https://de.wikipedia.org/wiki/ID3-Tag und Erklärung Blog „ID3 Tags Explained“.

Ausgabe einer MP3 Datei mit id3v2; hier mit Tags der Version 2:

```
id3v2 tag info for 42.0.mp3:
Title  : Ansage der Zahl 42.0           Artist: Thomas Wenzlaff
Album  : wenzlaff.de                    Year: 2018, Genre: Noise (39)
Comment: 42.0                           Track: 1
id3v2 tag info for 42.0.mp3:
TIT2 (Title/songname/content description): Ansage der Zahl 42.0
TRCK (Track number/Position in set): 1
TALB (Album/Movie/Show title): wenzlaff.de
TPE1 (Lead performer(s)/Soloist(s)): Thomas Wenzlaff
TYER (Year): 2018
COMM (Comments): ()[]: 42.0
TCON (Content type): Noise (39)
```

Hinweis: Passen Sie die Werte ggf. an die tatsächliche Ausgabe auf Ihrem Rechner an. Entscheidend ist, dass der gesprochene Zahlenwert sowohl im Dateinamen als auch in den Tags (z. B. `TIT2/Title`, `TRCK/Tracknummer`) kodiert ist.

Genau diese Kodierung macht die Dateisammlung zu einem guten Übungsobjekt für Shellskripte: Sie können per Skript gezielt nach Zahlenwerten suchen, Dateien filtern, umbenennen oder zu neuen Ausgaben kombinieren, ohne jede Datei einzeln manuell öffnen zu müssen. Stellen Sie sicher, dass Sie das Tool `id3v2` in der VM oder auf Ihrem System installiert haben (z.B.: `sudo apt install id3v2`).

Ihre Aufgabe: Schreiben Sie das bash Script „recode_id3.sh“

Sie schreiben ein Script „recode_id3.sh“ welches alle MP3 Dateien (mit Dateiendung .mp3 und .MP3) entweder:

- auflistet, sollten diese keine ID3v2 Tags enthalten, **oder**
- Von ID3v1 zu ID3v2 konvertiert.

Das Script sollte also mit dieser Beschreibung beginnen:

```
#!/bin/bash
#########################################################################
# Sample solution for Lab 1 exercise "recode_id3.sh"
# Usage:
#   recode_id3.sh --list-v1 [FILES]
#   recode_id3.sh --convert [FILES]
# Lists files using ID3 version 1 and Converts to version 2 tags.
# If no option is given, --list-v1 is the default.
# If FILES are not provided, search for .mp3 and .MP3 files subdirectories.
#
# Author: IHR NAME, HS Esslingen
# Date: 2026-09-30
#########################################################################
```

Mittels Parameter `--list-v1` werden die Dateien in FILES aufgeführt, die keine ID3v2 Tags beinhalten. Mittels Parameter `--convert` werden alle Tags in Dateien in FILES durch das Programm id3v2 konvertiert. Sollten keine FILES dem Script übergeben werden, soll das Script in allen Verzeichnissen und Unterverzeichnissen nach MP3-Dateien suchen. Über diese beiden Parameter hinaus sollte noch der Parameter `--help` implementiert werden, der diesen obigen Text auf der Konsole ausgibt und das Script dann beendet.

Bitte beachten Sie, dass Sie nach einer Konvertierung erst wieder die MP3s aus dem ZIP entpacken müssen. Der Fall, dass eine MP3-Datei überhaupt keine ID3-Tags enthält ist nicht abzufangen. Dateinamen mit einem Whitespace im Dateinamen sind ebenfalls nicht zu berücksichtigen.

Ihr Script muss sauber programmiert sein: dokumentiert durch Kommentare, Überprüfen möglicher Fehler und natürlich auch sauber formatiert.

Profi-Tipp:

Leerzeichen können in Shell-Scripten Ärger machen. Es hilft in diesen Fällen, wenn man einzelne Variable in Anführungszeichen setzt, also `"${f}"` statt `${f}`. Die Schreibweise für den Zugriff auf die Variable `$f` sollte auch wegen der Leerzeichen immer mit Klammern erfolgen, d.h. `${f}`.
Für das Überprüfen von `if`-Bedingungen in Shell hilft das Programm `test`, bspw. kann man den ersten Parameter $1 prüfen ob dieser leer ist mit:

```
if test "x${1}x" = "xx" ; then echo "empty first parameter" ; fi
```

Oder aber auch negiert durch Ausrufezeichen mit:

```
if ! test "${1}" ; then echo "empty first parameter" ; fi
```

Reflexionsfragen:

- Wie haben Sie ihr Bash-Script strukturiert (z.B.: Funktionen, Modularisierung, etc.)?
- Wie stellen Sie sicher, dass der Benutzer ihr Bash-Script korrekt einsetzt?
- Wie können Sie die Ausgabe auf der Konsole strukturieren (z.B.: wichtige Informationen hervorheben)?